

Hamara-Rozgar (RozgarOrch)

AI Service Orchestrator for Informal Economy

Google Antigravity Hackathon — Challenge 2

Author: Muhammad Ammar Asad

Framework: Google Antigravity & Firebase Cloud Database

Date: May 21st, 2026

Version: v1.0.0 (Production Release)

1. Executive Summary & Design Philosophy

Hamara-Rozgar (RozgarOrch) is an agentic, AI-driven marketplace orchestrator designed to solve fragmentation in Pakistan's informal economy. Informal sector jobs (plumbers, electricians, tutors, AC technicians, beauticians, mechanics) represent the backbone of Pakistan's labor market but lack centralized systems for scheduling, pricing verification, booking tracking, and dispute management.

This project introduces a highly coordinated, multi-agent AI system built on **Google Antigravity**. The application enables users to request services using natural conversational Urdu, Roman Urdu, or English. The orchestrator automatically extracts semantic parameters, resolves custom locations on-the-fly using geocoding engines, matches providers using a comprehensive 6-factor weight utility, generates dynamic travel-adjusted billing quotes, tracks job execution in real-time, and handles post-booking anomalies (such as en-route cancellations) automatically.

Core Success Criteria Achieved:

- **Multilingual Natural Language Parsing:** Fluid handling of Urdu script and Roman Urdu slang triggers.
- **Startup Geolocation Lock:** Instant browser Geolocation GPS acquisition on app startup.
- **Dynamic Custom Overrides:** Real-time geocoding for manual, non-pre-fetched sector queries.
- **Self-Healing Protocols:** Automated re-scheduling, reputation indexing, and client voucher compensations.

2. Decoupled Multi-Agent Architecture

The core orchestration layer implements a decoupled multi-agent cooperative architecture. The agents communicate parameters sequentially and react dynamically to operational state updates:

1. IntentAgent (Multilingual Context & Memory Parser)

Extracts intent parameters across Urdu, Roman Urdu, and English. When the live Gemini API key is provided, the agent queries Google AI Studio's `gemini-1.5-flash` model configured with structured JSON outputs. It preserves multi-turn conversation logs: if a user types a location refinement (e.g. *"main sector 4 airport society mein rehta hu"*) after requesting a service category, the IntentAgent merges the context cleanly. An offline regex-based fallback engine is built-in for zero-downtime resilience.

2. DiscoveryAgent (Location & Workload Balancer)

Manages provider lookup grids. Automatically locks to browser GPS coordinates on launch. If a custom address is specified, the DiscoveryAgent issues a live call to the **Google Geocoding API** to lock its Lat/Lng coordinates. It filters matching categories and calculates distance vectors using the Haversine formula.

Providers are ranked using a 6-factor utility function:

$$\text{Score} = (\text{Distance} * 0.25) + (\text{Rating} * 0.20) + (\text{Reliability} * 0.20) + (\text{Price} * 0.15) + (\text{Cancellation Policy} * 0.10)$$

3. PricingAgent (Dynamic Billing Engine)

Generates fair cost quotes for both the client and provider. It calculates:

- **Base Rate:** Provider's standardized baseline service fee.
- **Travel Allowance:** Calculated dynamically relative to coordinates (50 PKR per kilometer).
- **Urgency Surcharge:** +30% for immediate, high-severity requests.
- **Surge Surplus:** +15% capacity surge if active providers in the specialization are low.
- **Loyalty Discount:** -10% deduction to incentivize repeat bookings.

4. BookingAgent (Persistent Ledger)

Commits transactional bookings to persistent storage. Integrates natively with **Firebase Cloud Firestore**. If database connection is pending, the BookingAgent routes entries to a reactive local cache storage to prevent app interruption.

5. DisputeAgent (Self-Healing Fallback Engine)

Handles en-route anomalies. If an en-route provider cancels, the DisputeAgent immediately scans the discovery matching grid, re-assigns the job card to the next-best provider, issues an automated 150 PKR voucher credit to the client, and penalizes the cancelling provider's reliability index.

3. Design System & High-End Aesthetics

The application implements a premium, high-fidelity design system utilizing responsive Glassmorphic rules built completely on Vanilla CSS:

- **Smartphone Simulator:** A premium, interactive frame showcasing the chat feed, quick preset Urdu slang chips, post-booking en-route tracker, and dispute simulator options.
- **Proximity Banner:** A glassmorphic location widget beneath the app header that expands to toggle GPS modes and resolve geocoded inputs, showing pulsing green and blue status lights.
- **Antigravity Trace Console:** A live developer logger displaying active workplans, task lists, sub-agent reasoning processes, and API triggers.
- **Marketplace Control Center:** A panel to coordinate active provider balances and configure active Firebase, Gemini, and Google Maps API keys dynamically.

4. API Integration & Payload Schemas

The platform integrates with three primary cloud services:

1. Gemini Intent Generation Endpoint

```
POST https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=
Content-Type: application/json
```

```
{
  "contents": [
    { "role": "user", "parts": [{ "text": "meri motorcyle khrab ho gyi hai" }] },
  ],
  "system_instruction": {
    "parts": [{ "text": "Parse natural language to structured service/location JSON..." }]
  },
  "generationConfig": {
    "responseMimeType": "application/json"
  }
}
```

2. Google Geocoding Endpoint

```
GET https://maps.googleapis.com/maps/api/geocode/json?address={LocationName},Islamabad,Pakistan&key=
```

3. Firebase Firestore Booking Schema

```
{
  "id": "BK-3914",
  "providerName": "Ali AC Services",
  "service": "AC Technician",
  "location": "GPS Location",
  "locationCoords": { "latitude": 33.6409, "longitude": 72.9814 },
  "pricing": { "totalPrice": 1650 },
  "status": "Provider En-Route"
}
```

5. Compilation & Standalone APK Packaging

The project is fully pre-configured to be packaged into a native, standalone installer Android APK using the **Capacitor** framework.

To compile the native app bundle on your local development machine with a standard Java JDK and Android SDK setup, execute the automated helper scripts provided in the root of the project:

- **On macOS/Linux:** Open your terminal and run `./build_apk.sh`.
- **On Windows:** Double-click the file `build_apk.bat`.

The builder will compile your React files, synchronize native resources, and invoke the Gradle wrapper task:

```
cd android && ./gradlew assembleRelease
```

The compiled debug/release installer APK will be generated immediately under:
`android/app/build/outputs/apk/release/app-release-unsigned.apk`